Section 13

Section 12

Functions and Prime Numbers

Topics: functions, parameters, pass by reference, struct
Lab 4: https://maryash.github.io/135/labs/lab_05.html

Ryan Vaz

What are functions?

A function is a named sequence of instructions that performs a specific task and returns the result of its computation. Once defined, it can be then called in your program wherever that particular task should be performed.

A function can receive zero or more arguments. For example, consider a function sum, which receives three arguments named a, b, and c, and returns their sum:

```
int sum(int a, int b, int c) {
    return a + b + c;
}

int main(){
    cout << sum(2,3,4) << endl; // prints "9" which is the sum of 2, 3, and 4
}
```

Function Declaration

The function declaration, also known as the function prototype, contains the name of the function, what type of data the function returns and what type of data the function takes. Let's look at the sum function:

```
int sum(int a, int b, int c);
```

From the function declaration, we know the following:

- Function name: sum()
- Return type: integer
- Parameters: three integer parameters passed *by value*

Function Declaration

The arguments given to a function are known formally as parameters. Each parameter contains a data type followed by a name. There can be functions that take no parameters. For example:

```
void sayHi(){
    cout << "Hi!" << endl;
}
```

If a parameter is passed by value, it can be used like a variable:

```
int multiply(int a, int b){
    int returnValue = a * b;
    return returnValue;
}
```

Declaration vs Definition

Similar to variables, we can declare a function without initializing it.

When we do this, we make a promise to the compiler that later on we will write a definition for the function.

In general C++ is read from top to bottom. If we don't have a declaration at the top, the compiler will complain about the function being undefined.

```
int multiply(int a, int b);

int main(){
    cout << multiply(9, 2);
}

int multiply(int a, int b){
    int returnValue = a * b;
    return returnValue;
}
```

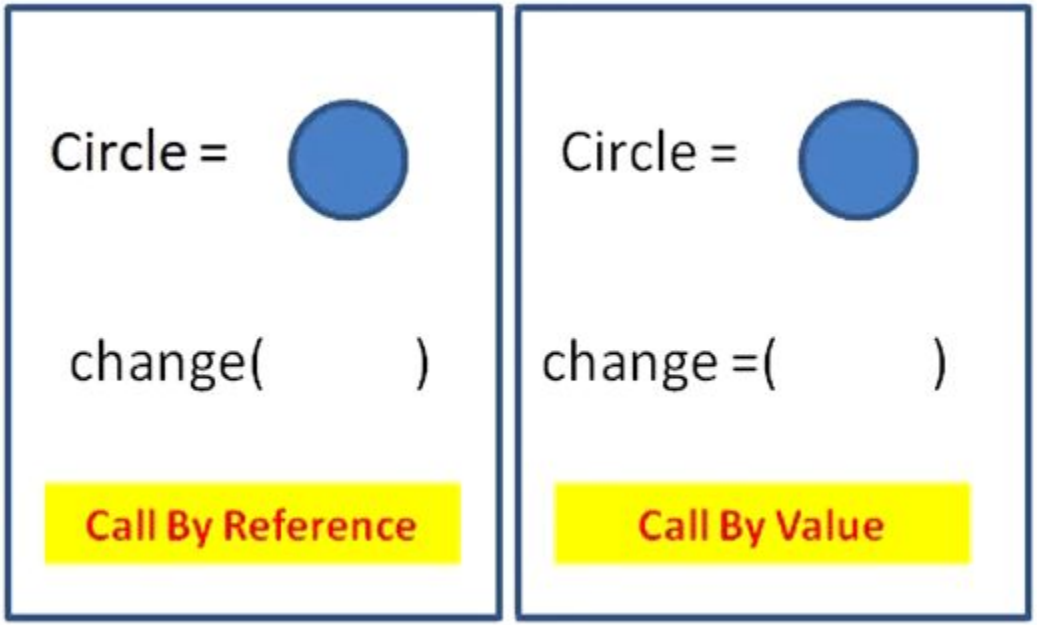
Passing parameter by reference

So far, the examples showed the parameter being passed by value. There are cases where you want to modify a given parameter so that it changes the actual value of the given data. In such cases, parameters can be passed by reference(&):

```
void increment(int &a) {                    // a is passed by reference using & operator
    a++;                                    // increment given parameter by 1
}
int main(){
    int x = 2;                              // declare variable x as 2
    cout << "Before: " << x << endl;        // prints "Before: 2"
    increment(x);                           // increment variable x by 1
    cout << "After: " << x << endl;         // prints "After: 3"
}
```

The value of the variable x was changed after calling the function!

Pass by reference visualized



Passing parameters as `const`

Passing large data by value can be costly. Therefore passing by reference is more space efficient since we don't need to make a copy of the object.
What if you want to pass something by reference and you don't want the function to modify the parameter? In such cases, use the const keyword to pass a parameter by reference while ensuring that the function won't modify the data that it gets. The parameter is passed as a read-only reference:

```
void increment(const int &a) { // a is passed as read-only reference
    a++;                       // THIS WILL THROW AN ERROR BECAUSE `a` IS const
}
int main(){
    int x = 3;
    increment(x);              // leads to an error from increment function
}
```

Returning from a function

The data being returned must match with the data promised by the function declaration. If the function declaration says it returns an integer, it must return an integer! If it doesn't, compiler will complain:

```
int func() {     // func promises to return an integer but returns string instead
    return "1"; // ERROR SINCE FUNCTION RETURNS STRING FROM INTEGER FUNCTION
}
int main() {
    func();      // WILL CAUSE AN ERROR BECAUSE func() DOESN'T RETURN AN INTEGER
}
```

The main function is an exception. If you don't return anything from main(), it returns 0 by default. Some compilers will give a warning for non-returning main() functions.

Returning to exit a void function

Void functions can also `return`. However, no value is returned since it is void. The `return` serves as a way to exit the function early.

```
void func(){
    cout << "before returning" << endl;
    return;                                    // exit out of the function
    cout << "You will never see this!" << endl; // never executed
}
int main(){
    func();                                    // prints "before returning"
}
```

Returning any value from a void function will cause a compilation error.

Parallel arrays

When we want to store data about a specific value that contains multiple primitive types, we can model this by using parallel arrays. In a parallel array, we have multiple arrays where one index correlates to talking about the same object. A parallel array is composed of multiple arrays using primitive data types (strings, ints, doubles, etc).

```
string names[] = ["Sparky", "Jenna", "Hachiko", "Kira"];
string breeds[] = ["Poodle", "Chihuahua", "Akita", "Husky"];
int ages[] = [2,1,7,5];

cout << names[2] << " is a " << breeds[2] << " who is " << ages[2] << " years old." << endl;
// 2 -  Hachiko is an Akita who is 7 years old.
// Everything at index i corelates to the same dog!
```

You've probably already used this before with Project 1B, this is just a formalization of the concept. `questions` and `answers` were parallel arrays in this segment.

Structures

A structure (struct) is a user defined data type that allows us to combine multiple primitive data types into a single new data type.

Once you have the structure defined, you can declare it like any old variable. When the struct is initialized, all the *data members* will be empty until we directly give them values. We can do so by using the . (dot) operator, which allows us to access members of a struct. Alternatively we can initialize it in one line using the below syntax.

```
struct Dog{
    string name;
    string breed;
    int age;
};

Dog myDog;

myDog.name = "Hachiko";
myDog.breed = "Akita";
myDog.age = 7;
```

This concept will be utilized in Project 1C heavily!

```
Dog hachiko = {"Hachiko", "Akita", 7};
```